Documentação de Requisitos de Software (SRS)

Arquitetura Multi-Linguagem para E-commerce · Recorte acadêmico

1. Visão Geral da Arquitetura e Matriz de Tecnologias por Módulo

O ecossistema do e-commerce é estruturado em uma arquitetura orientada a serviços com banco de dados relacional centralizado (**PostgreSQL 15+**). A divisão de responsabilidades tecnológicas é estritamente delimitada para garantir isolamento de domínios, alta performance de leitura e segurança transacional.

Front: a **loja** é **Angular**. O **admin** é **PHP (Laravel)**. As duas UIs não se misturam.

Node: um único processo Express (TypeScript) — identidade, catálogo e WebSocket. **Não há Fastify.**

Módulo do Sistema	Linguagem / Runtime / Framework	Papel Arquitetural e Escopo Operacional
Front da Loja (consumidor)	**Angular**	SPA: vitrine, PDP, carrinho, checkout e "meus pedidos". Consome Express (auth/catálogo) e Java (pedido/pagamento).
Módulo de Autenticação, Catálogo e Tempo Real	**Node.js (TypeScript / Express)** — um processo	Único emissor de JWT; BFF da vitrine (`JSONB` /GIN); WebSocket. Endpoint interno `POST /internal/order-status` recebe aviso do PHP.
Módulo Core de Transações e Checkout	**Java (Spring Boot / Jakarta EE)**	Pedidos autenticados, **gateway de pagamento simulado**, concorrência de estoque e funções PL/pgSQL.
Módulo de Retaguarda / ERP (Background)	**Java RMI**	Dois jobs demonstráveis: consolidação de estoque e fechamento de caixa. Sem UI; efeito na auditoria e no dashboard.
Módulo de Painel Administrativo (Backoffice)	**PHP (Laravel / PDO)**	Só admin: catálogo, categorias, estoque, pedidos, cadastro de operadores e auditoria. Ao mudar status, chama o HTTP interno do Node.

2. Módulo 1: Autenticação, Cadastro e Perfil de Usuários

- **Tecnologia Responsável: Node.js (TypeScript / Express)** (mesmo processo da seção 3).
- **Regra de Banco de Dados:** Tabela `ecommerce.users` com domínio `CITEXT` para e-mails e unicidade de documentos (CPF/CNPJ).
- **Papéis (RBAC mínimo):** apenas `CLIENTE` e `ADMIN`.
- **Contrato de identidade (obrigatório nos três runtimes):**

- O Express é o **único emissor** do JWT (Bcrypt em `password_hash`; claims `sub` = `user_id`, `role`, `context`).
- Express (rotas autenticadas / WebSocket), Java (checkout) e Laravel (backoffice) **validam** o mesmo token.
- `context=STORE` libera loja e checkout; **não** abre o PHP.
- `context=ADMIN` libera o Laravel; **não** fecha pedido na loja.
- Cliente autenticado cai na loja (`STORE`). Admin autenticado vê a tela de escolha de contexto.
- Pedidos (`ecommerce.orders.user_id`) e `audit.activity_log` gravam o `sub`. Sem `user_id` a operação é rejeitada.
- Checkout **exige** login. Não há compra como convidado.

2.1. Tela de Login (`/auth/login`)

- **Especificação Funcional:** Login unificado para clientes e administradores.
- **Campos da Interface:**
  - `input_email`: E-mail (Exemplo: `cliente.suporte@dominio-exemplo.com`).
  - `input_password`: Senha oculta.
  - `link_forgot_password`: Atalho para `/auth/forgot`.
  - `btn_submit`: POST para a API Express.
- **Comportamento e Regras:** Valida Bcrypt em `password_hash` e emite JWT. `role=CLIENTE` → `/catalog/home` com `context=STORE`. `role=ADMIN` → `/auth/context`.

2.2. Tela de Cadastro de Cliente (`/auth/register`)

- **Especificação Funcional:** Formulário público. Cria **somente** `CLIENTE`. Ninguém se promove a `ADMIN` aqui.
- **Campos da Interface:**
  - `input_fullname`: Nome completo (mínimo 3 caracteres).
  - `input_email`: E-mail único (Exemplo: `novo.usuario.teste@provedor-exemplo.org`).
  - `input_password` / `input_confirm_password`: Senha e confirmação.
  - `input_document`: CPF (11 dígitos) ou CNPJ (14 dígitos), Regex no banco.
  - `input_phone`: Telefone E.164 (Exemplo: `+5561988887777`).
- **Comportamento e Regras:** Insert dispara trigger `audit.activity_log`. Após sucesso, autentica e envia à vitrine.

2.3. Tela de Esqueci a Senha (`/auth/forgot`)

- **Especificação Funcional:** Pedido de redefinição por e-mail.

- **Campos da Interface:**
  - `input_email`: E-mail cadastrado.
  - `btn_submit`: Gera token de reset com expiração.
- **Comportamento e Regras:** Não revela se o e-mail existe. Em ambiente acadêmico o link pode ir para log/console, sem SMTP. O token de reset **não** é o JWT de sessão.

2.4. Tela de Redefinir Senha (`/auth/reset/:token`)

- **Especificação Funcional:** Nova senha a partir do token.
- **Campos da Interface:**
  - `input_password` / `input_confirm_password`.
  - `btn_submit`: Atualiza `password_hash` (Bcrypt) e invalida o token.
- **Comportamento e Regras:** Token expirado ou usado → erro e retorno a `/auth/forgot`. Sucesso → `/auth/login`.

2.5. Tela de Escolha de Contexto (`/auth/context`)

- **Especificação Funcional:** Só `role=ADMIN`. Cliente comum nunca vê esta tela.
- **Componentes da Interface:**
  - `btn_context_store`: Reemite JWT `context=STORE` → `/catalog/home`.
  - `btn_context_admin`: Reemite JWT `context=ADMIN` → `/admin/dashboard`.
- **Comportamento e Regras:** Sem a claim `context` correspondente, Java recusa checkout e PHP recusa o painel.

2.6. Encerramento de Sessão (Logout)

- **Especificação Funcional:** Ação no cabeçalho da loja e do painel (`POST /auth/logout`).
- **Comportamento e Regras:** Invalida o token (`jti` / deny-list). Angular/PHP descartam o JWT e redirecionam para `/auth/login`.

3. Módulo 2: Catálogo de Produtos e Busca Avançada

- **Tecnologia Responsável: Node.js (TypeScript / Express)** — mesmo processo da seção 2.
- **Front consumidor: Angular**.
- **Regra de Banco de Dados:** Consultas em `ecommerce.products`, `ecommerce.categories` e `ecommerce.inventory` com índice GIN no `JSONB`.
- **Acesso:** vitrine, busca e PDP são **públicas**. O carrinho fica no Angular até o checkout; o Java recebe os itens no `POST` autenticado de pedido.

- **Tempo real:** o PHP, ao transicionar status em `/admin/orders`, chama `POST /internal/order-status` no Express (rede interna, sem JWT de usuário; payload `orderId`, `status`). O Node empurra via WebSocket para o Angular em `/account/orders`. Sem essa chamada, o cliente só vê o status no próximo refresh.

3.1. Tela de Vitrine / Home (`/catalog/home`)

- **Especificação Funcional:** Produtos em destaque e categorias principais.
- **Componentes da Interface:**
  - `search_bar`: Busca textual indexada.
  - `category_tree_menu`: Árvore `parent_id`.
  - `product_grid`: Imagem, nome, SKU e preço `NUMERIC(12,2)`.

3.2. Tela de Listagem com Filtros Dinâmicos (`/catalog/search`)

- **Especificação Funcional:** Resultados com filtros facetados.
- **Componentes da Interface:**
  - `sidebar_filters`: Chaves do `JSONB` `attributes` (voltagem, cor, dimensões).
- **Comportamento e Regras:** `WHERE attributes @> '{"voltagem": "220v"}'` (sem sequential scan).

3.3. Tela de Detalhes do Produto - PDP (`/catalog/product/:slug`)

- **Especificação Funcional:** Ficha técnica do item.
- **Componentes da Interface:**
  - `tech_specs_table`: Pares chave-valor do `JSONB`.
  - `stock_badge`: Disponibilidade em `ecommerce.inventory`.
  - `btn_add_to_cart`: Inclui o SKU no carrinho Angular e oferece `/checkout/cart`.

4. Módulo 3: Carrinho, Checkout, Pedidos do Cliente e Pagamento

- **Tecnologia Responsável: Java (Spring Boot / Jakarta EE)**
- **Front: Angular** (mesmas rotas abaixo).
- **Regra de Banco de Dados:** `@Transactional` nas funções `ecommerce.reservar_estoque_pedido` e `ecommerce.confirmar_baixa_estoque` com `FOR UPDATE`. HTTP de gateway **não** entra nessa transação.
- **Autorização:** `/checkout/*` e `/account/orders*` exigem JWT com `role=CLIENTE|ADMIN` e `context=STORE`. O `user_id` do pedido é o `sub`, nunca campo do cliente.
- **Contrato do carrinho:** sem sessão HTTP Node↔Java. O Angular envia o snapshot de itens no `POST /api/v1/orders`. Daí o Java é a fonte da verdade.

- **Máquina de estados (única):** `PENDING` → `PAID` → `PROCESSING` → `SHIPPED` → `DELIVERED`. `CANCELED` é terminal a partir de `PENDING`, `PAID` ou `PROCESSING`.

4.1. Tela de Carrinho de Compras (`/checkout/cart`)

- **Especificação Funcional:** Revisão de itens antes do fechamento.
- **Componentes da Interface:**
  - `cart_items_table`: Quantidade, preço unitário congelado, subtotal.
  - `btn_proceed_checkout`: Sem login → `/auth/login` e volta ao carrinho. Com `context=STORE` → `/checkout/address`.
- **Fora de escopo:** cupom/código promocional. Não há campo na UI nem CRUD no admin.

4.2. Tela de Seleção de Endereço de Entrega (`/checkout/address`)

- **Especificação Funcional:** Destino da carga, vinculado à conta.
- **Componentes da Interface:**
  - `address_radio_group`: `ecommerce.addresses`, destaque `is_default = TRUE`.
  - `new_address_form`: CEP 8 dígitos, UF maiúsculo.

4.3. Tela de Pagamento e Fechamento (`/checkout/payment`)

- **Especificação Funcional:** Submissão do pedido e meio de pagamento.
- **Componentes da Interface:**
  - `financial_summary`: Subtotal, frete e `total_amount` (coluna computada). Sem linha de desconto.
  - `payment_method_selector`: Enum `PIX`, `CREDIT_CARD`, `BOLETO`.
  - `btn_submit_order`: Fechamento.
- **Comportamento e Regras:**
  1. O **cliente** envia `idempotency_key` estável por tentativa. O Java deduplica por essa chave.
  2. Reserva de estoque no PostgreSQL (`FOR UPDATE`) para impedir *overselling*.
  3. **Frete:** regra simples no backend (fixo ou isenção acima de um limiar). Sem tela PAC/SEDEX. A UI só mostra a linha no resumo.
  4. **Gateway simulado:** o Java **não** chama Stripe/Mercado Pago. Adaptador interno persiste o retorno em `gateway_payload` (`JSONB`) e define `PENDING` (PIX/boleto) ou `PAID` (cartão aprovado no mock).

4.4. Tela de Confirmação do Pedido (`/checkout/success/:orderId`)

- **Especificação Funcional:** Recibo e instruções de pagamento.

- **Componentes da Interface:**
  - `order_status_badge`: `PENDING` ou `PAID`.
  - `pix_qrcode_container` / `boleto_barcode`: gerados a partir de `gateway_payload`.
  - `link_my_orders`: `/account/orders`.

4.5. Tela Meus Pedidos (`/account/orders`)

- **Especificação Funcional:** Histórico do cliente. Destino do WebSocket de status.
- **Componentes da Interface:**
  - `orders_table`: Pedidos do `user_id` do token.
  - `status_badge`: Estado da máquina única.

4.6. Tela Detalhe do Pedido do Cliente (`/account/orders/:orderId`)

- **Especificação Funcional:** Itens, endereço, pagamento e linha do tempo.
- **Comportamento e Regras:** Só se `orders.user_id = sub`. Status ao vivo via WebSocket (evento originado pelo `POST /internal/order-status` do PHP).

5. Módulo 4: Painel Administrativo (Backoffice)

- **Tecnologia Responsável: PHP (Laravel / PDO)**
- **Regra de Banco de Dados:** PgBouncer, usuário `svc_php_web`, `PDO::ATTR_EMULATE_PREPARES => false`.
- **Autorização:** JWT com `role=ADMIN` e `context=ADMIN`. Sem token → `/auth/login`. Mutações gravam `user_id` em `audit.activity_log`.
- **Bootstrap:** o primeiro `ADMIN` nasce por seed. Novos admins só na tela 5.6.

5.1. Tela de Dashboard Operacional (`/admin/dashboard`)

- **Especificação Funcional:** Métricas de faturamento (inclui o resumo gravado pelo job RMI de fechamento de caixa).
- **Componentes da Interface:**
  - `revenue_chart`: Faturamento diário.
  - `orders_status_counters`: `PENDING`, `PAID`, `PROCESSING`, `SHIPPED`, `DELIVERED`, `CANCELED`.

5.2. Tela de Gestão de Catálogo (`/admin/products`)

- **Especificação Funcional:** CRUD de mercadorias (cadastro e preço). Saldo **não** é editado aqui.
- **Componentes da Interface:**
  - `products_datatable`: Busca por SKU ou nome.

- `product_form`: Preço `NUMERIC(12,2)`, categoria filha, atributos `JSONB`.

5.3. Tela de Gestão de Categorias (`/admin/categories`)

- **Especificação Funcional:** CRUD da árvore `ecommerce.categories` (`parent_id`).
- **Componentes da Interface:**
  - `category_tree`: Pai / filha.
  - `category_form`: Nome, slug, pai opcional.
- **Comportamento e Regras:** Não excluir categoria com produtos vinculados.

5.4. Tela de Estoque / Inventário (`/admin/inventory`)

- **Especificação Funcional:** Tela **própria** sobre `ecommerce.inventory`.
- **Componentes da Interface:**
  - `inventory_datatable`: SKU, disponível, reservado.
  - `stock_adjust_form`: Entrada/saída com motivo; auditoria com `user_id`.
- **Comportamento e Regras:** Ajuste manual não substitui as funções Java de reserva/baixa. A PDP lê esta tabela. O job RMI de consolidação também escreve aqui.

5.5. Tela de Gestão de Pedidos e Logística (`/admin/orders`)

- **Especificação Funcional:** Ciclo de vida dos pedidos.
- **Componentes da Interface:**
  - `orders_table`: Filtros por data e status.
  - `action_buttons`: `PAID` → `PROCESSING` → `SHIPPED` → `DELIVERED`; cancelamento quando aplicável.
- **Comportamento e Regras:** Cada transição registra o operador na auditoria e dispara `POST /internal/order-status` no Express.

5.6. Tela de Cadastro Interno de Administrador (`/admin/users`)

- **Especificação Funcional:** Cria `ADMIN`. Não é pública.
- **Campos da Interface:**
  - `input_fullname`, `input_email`, `input_password`, `input_document`, `input_phone` (regras da tela 2.2).
  - `chk_can_shop`: Se marcado, o admin também compra na loja (tela 2.5).
- **Comportamento e Regras:** Insert pela API de identidade Express, nunca por `/auth/register`.

5.7. Tela de Auditoria Forense do Sistema (`/admin/audit-logs`)

- **Especificação Funcional:** Somente leitura de `audit.activity_log` (inclui linhas `app_name = rmi` dos jobs).
- **Componentes da Interface:**
  - `audit_datatable`.
  - `filter_panel`: tabela, operação (`INSERT`/`UPDATE`), IP, `user_id`, `app_name`.
  - `json_diff_viewer`: `old_data` vs `new_data`.

6. Módulo 5: Sistema de Retaguarda / ERP (Background Processing)

- **Tecnologia Responsável: Java RMI**
- **Regra de Banco de Dados:** JDBC + HikariCP, credencial `svc_java_rmi`.

6.1. Processamento de Background (Sem Interface Gráfica)

- **Jobs demonstráveis na banca (obrigatórios):**
  1. **Consolidação de estoque:** reconcilia `ecommerce.inventory` (disponível vs reservado) e grava `audit.activity_log` com `app_name = rmi`. Cron + disparo manual ("rodar agora") na demo.
  2. **Fechamento de caixa / consolidação fiscal:** agrega pedidos `PAID`/`DELIVERED` do dia, persiste resumo interno e alimenta o gráfico do dashboard PHP.
- **Como a banca vê o RMI:** não há tela RMI. O efeito aparece na **auditoria** e no **dashboard**. A demo dispara o job na hora.
- **Fora de escopo:** frete PAC/SEDEX; fornecedor terceiro real.

7. Inventário de Telas

#	Rota	Quem	Acesso
2.1	`/auth/login`	Angular → Express	Público
2.2	`/auth/register`	Angular → Express	Público (só `CLIENTE`)
2.3	`/auth/forgot`	Angular → Express	Público
2.4	`/auth/reset/:token`	Angular → Express	Com token
2.5	`/auth/context`	Angular → Express	Só `ADMIN`
2.6	Logout	Angular/PHP → Express	Autenticado
3.1	`/catalog/home`	Angular → Express	Público
3.2	`/catalog/search`	Angular → Express	Público

#	Rota	Quem	Acesso
3.3	`/catalog/product/:slug`	Angular → Express	Público
4.1	`/checkout/cart`	Angular → Java	Checkout exige login
4.2	`/checkout/address`	Angular → Java	`context=STORE`
4.3	`/checkout/payment`	Angular → Java	`context=STORE`
4.4	`/checkout/success/:orderId`	Angular → Java	`context=STORE`
4.5	`/account/orders`	Angular → Java/Express	`context=STORE`
4.6	`/account/orders/:orderId`	Angular → Java/Express	`context=STORE`
5.1	`/admin/dashboard`	PHP Laravel	`context=ADMIN`
5.2	`/admin/products`	PHP Laravel	`context=ADMIN`
5.3	`/admin/categories`	PHP Laravel	`context=ADMIN`
5.4	`/admin/inventory`	PHP Laravel	`context=ADMIN`
5.5	`/admin/orders`	PHP Laravel	`context=ADMIN`
5.6	`/admin/users`	PHP Laravel	`context=ADMIN`
5.7	`/admin/audit-logs`	PHP Laravel	`context=ADMIN`
6.1	—	Java RMI	Sem UI

8. Decisões de recorte (fechadas)

- **Loja:** Angular. **Admin:** PHP Laravel.
- **Node:** um processo Express (auth + catálogo + WebSocket). Fastify fora.
- **PHP → Node:** `POST /internal/order-status`. Sem LISTEN/NOTIFY e sem fila.
- **Pagamento:** gateway **simulado** no Java; `gateway_payload` JSONB; sem Stripe/Mercado Pago.
- **RMI:** dois jobs (consolidar estoque + fechamento de caixa), visíveis na auditoria/dashboard.
- **Cupom:** fora de escopo.
- **Modelo de dados:** DDL/migrations no PostgreSQL na implementação; ERD no relatório final (não duplicado neste SRS).